Part 1 — Behavioral & Project Discussion

Chapter 1: Tell Me About Yourself

Probability: 🔴 99% — Asked in EVERY round

Why Interviewers Ask This

This is NOT a casual question. It is your **first impression audit**. The interviewer is evaluating: - Can you communicate clearly and concisely? - Do you understand your own career trajectory? - Are you a fit for this specific role? - Do you show confidence without arrogance?

The Formula (for 4 YOE DevOps)

```
[1] Hook (who you are) → 15 seconds
[2] Core experience (what you do) → 30 seconds
[3] Key achievements (proof of impact) → 30 seconds
[4] Why this role / why now → 15 seconds
```

Your Scripted Answer

> *"Hi, I'm Aniket. I'm a Senior DevOps Engineer with 4+ years of experience, currently working as a Technical Lead at Kochar Innovations, where I lead a DevOps team of 8+ engineers.*
>
> *My core expertise is in building and managing production infrastructure — CI/CD pipelines with Jenkins and GitLab CI, containerized workloads on Kubernetes and Docker, full-stack observability with Prometheus, Grafana, and Loki, and Infrastructure as Code with Terraform and Ansible.*
>
> *Some of my key contributions include containerizing 20+ microservices with a 60% image size reduction, architecting HA database platforms with Patroni and Redis Sentinel, driving cloud cost optimization that saved 25% monthly spend, and supporting ISO 27001 and SOC 2 compliance initiatives. I've managed 15+ production applications with 99.9% SLA.*
>
> *I'm now looking to take on a Senior DevOps role at Infosys because I want to work on larger-scale Azure and Kubernetes environments across multiple client projects, and I believe my infrastructure ownership and platform engineering experience directly aligns with what this role requires."*

Common Mistakes to AVOID

Mistake	Why It's Bad	Fix
Starting with "I was born in..."	Wastes time, unprofessional	Start with your current role
Listing every tool you've used	Sounds like reading a resume	Focus on 3-4 core strengths

Mistake	Why It's Bad	Fix
Being vague ("I do DevOps stuff")	Shows no depth	Use specifics: numbers, tools, impact
Speaking for 5+ minutes	Loses interviewer attention	Keep it under 90 seconds
Not mentioning the target role	Missed connection opportunity	End with why THIS role
Memorizing word-for-word	Sounds robotic	Know the structure, not the script

Follow-up Questions to Expect

1. **"Can you elaborate on your team lead responsibilities?"** → Emphasize: tech lead for 8+, direct management of 2-3 interns, sprint coordination, RCA ownership, incident coordination
2. **"Why are you leaving your current company?"** → "I'm looking for enterprise-scale projects with larger client exposure. Infosys offers the opportunity to work across multiple industries and cloud environments."
3. **"What's your biggest technical achievement?"** → Pick ONE. Go deep. Example: "Containerizing 20+ services with multi-stage Dockerfiles, reducing image sizes by 60% and deployment time by 50%, while implementing Grype-based security scanning that blocked 90% of CVE-exposed deployments."
4. **"What's your experience with Azure?"** → Be HONEST: "My production experience is primarily with AWS and OCI. However, I have strong conceptual understanding of Azure services — Azure DevOps maps to Jenkins+GitLab, AKS maps to my Kubernetes experience, Azure Monitor maps to Prometheus+Grafana. The underlying engineering principles are the same; the API surface is what changes."

Chapter 2: Current Project Discussion

Probability: 🔴 99% — Asked in EVERY technical round

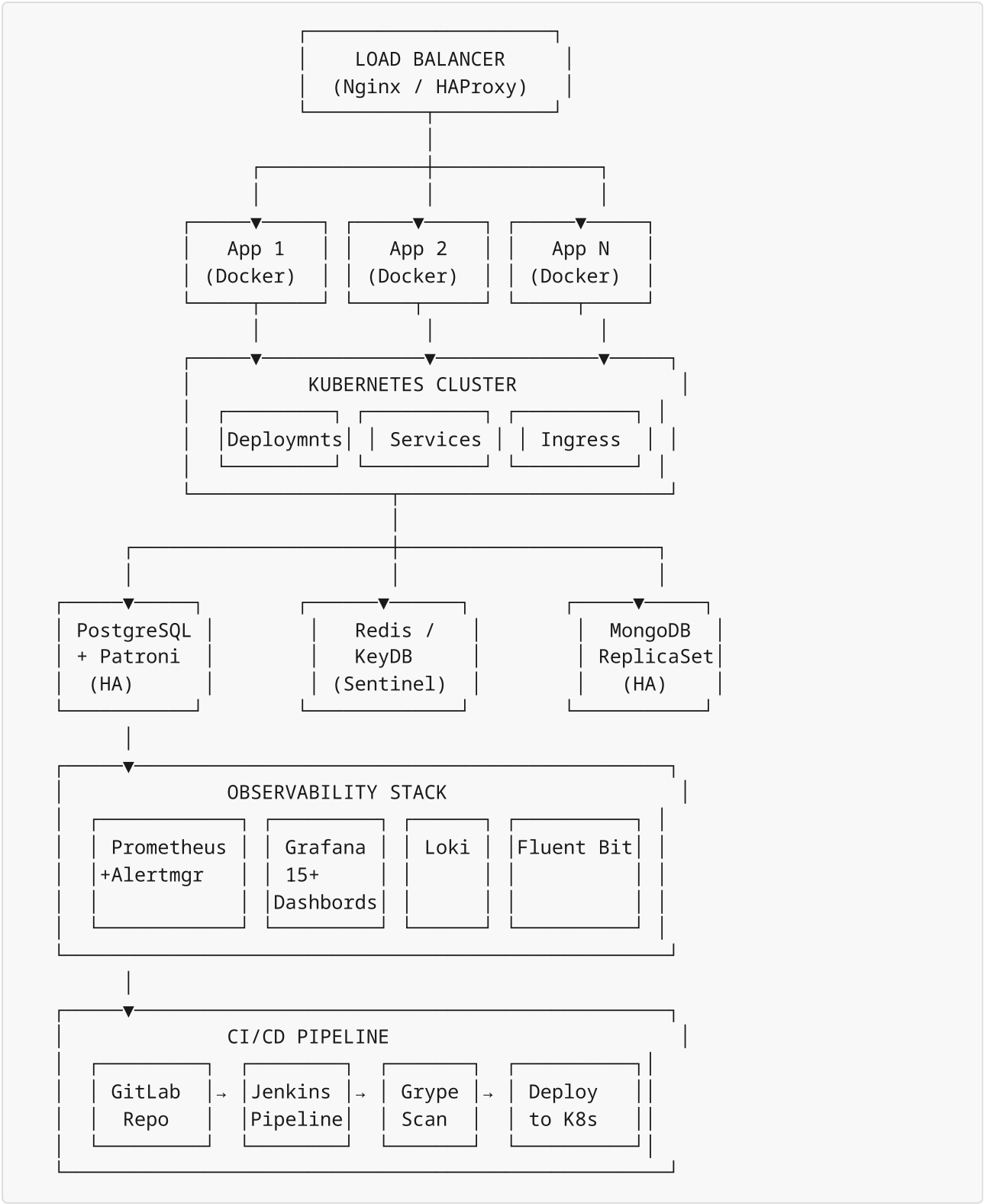
Why This Matters

At Infosys, interviewers **pick apart your current project** to assess: - How deeply you understand your own architecture - Whether you actually built it or just operate it - Your decision-making process - Your ability to explain complex systems simply

How to Structure Your Project Walkthrough

```
[1] Project context (what, who, why) → 30 seconds
[2] Architecture overview → 60 seconds
[3] Your specific contributions → 60 seconds
[4] Challenges & how you solved them → 60 seconds
[5] Impact / metrics → 30 seconds
```

Your Project Architecture (ASCII Diagram)



Sample Explanation

> *"I work at Kochar Innovations where we manage 15+ production applications across multi-cloud environments — primarily AWS and OCI, with some on-premise infrastructure.*
>
> *The architecture is microservices-based. We have 20+ containerized services running on Kubernetes, fronted by Nginx/HAProxy load balancers. Our data layer includes PostgreSQL with Patroni for automatic failover, MongoDB replica sets, Redis Sentinel for caching, and ClickHouse for analytics.*

> *For CI/CD, we use a combination of Jenkins and GitLab CI/CD. Code is pushed to GitLab, triggers a Jenkins pipeline that runs unit tests, builds multi-stage Docker images, runs Grype vulnerability scanning, and deploys to Kubernetes using rolling updates.*
>
> *For observability, I built the entire monitoring stack from scratch — Prometheus scrapes metrics from 100+ hosts and 200+ containers, Alertmanager routes alerts, Grafana has 15+ dashboards, and Loki+Fluent Bit handles centralized logging.*
>
> *My specific contributions include defining infrastructure standards and deployment patterns adopted across multiple application teams, implementing disaster recovery with automated snapshots and cross-region replication achieving sub-30-minute RTO, and driving cloud cost optimization that reduced spend by 25%."*

Deep-Dive Questions to Expect

Question	What They're Really Asking	How to Answer
"Why did you choose Patroni over pgpool?"	Do you understand HA tradeoffs?	"Patroni provides automatic leader election via etcd/consul, handles fencing properly, and integrates well with our existing tooling. pgpool is more of a connection pooler that also does HA, but Patroni is purpose-built for PostgreSQL HA."
"How do you handle database schema migrations?"	Do you think about data layer in CI/CD?	"We use migration tools (Flyway/Liquibase concepts) integrated into the deployment pipeline. Migrations run as a pre-deploy step, with backward-compatible changes first, then destructive changes in a separate release."
"What happens if your Jenkins master goes down?"	Do you design for resilience?	"We have Jenkins configured with periodic backup of JENKINS_HOME. For critical pipelines, we also have GitLab CI/CD as a fallback. Our infrastructure standards include documented recovery runbooks with tested RTO."
"How do you handle secrets?"	Security awareness	"Secrets are stored in vault/secret management, never in code. In K8s, we use Kubernetes Secrets (encrypted at rest). In CI/CD, credentials are managed through Jenkins credentials store with appropriate RBAC."
"Walk me through a deployment failure"	Incident response maturity	Use STAR format. Show you have a process, not just panic.

Common Mistakes

1. **Not knowing your own architecture** — If you say "PostgreSQL with Patroni" and can't explain how Patroni leader election works, you're in trouble
2. **Saying "the team did it"** — Use "I" for your contributions, "we" for team efforts. Own your work
3. **Not having metrics** — Always quantify. "We improved deployment speed" is weak. "Reduced deployment time from 45 minutes to 12 minutes" is strong
4. **Overcomplicating** — Don't try to make your project sound bigger than it is. Simple + deep > complex + shallow

Chapter 3: Daily Responsibilities

Probability: 🟠 80% — Frequently Asked

Why They Ask

They want to know if your daily work matches what they need. For a Senior DevOps role at Infosys, they need someone who can: - Handle production independently - Make infrastructure decisions - Coordinate across teams - Troubleshoot under pressure

How to Frame Your Daily Responsibilities

Morning Routine:

```
08:30 - Check monitoring dashboards (Grafana)
      - Review overnight alerts (Alertmanager/PagerDuty)
      - Check pipeline status (Jenkins/GitLab)
09:00 - Daily standup (Agile/Scrum)
      - Report blockers
      - Coordinate with dev teams
```

Core Work:

```
09:30-13:00 - Infrastructure work
           - Pipeline improvements
           - IaC changes (Terraform/Ansible)
           - Code reviews for infra changes
           - Deployment support
14:00-17:00 - Incident response (if any)
           - Sprint tasks
           - Documentation updates
           - Mentoring juniors/interns
           - Capacity planning
17:00-17:30 - End-of-day checks
           - Handoff to on-call if applicable
```

Interview Answer Template

> *"My typical day starts with checking our Grafana dashboards and Alertmanager for any overnight alerts. I review the CI/CD pipeline status in Jenkins and GitLab to catch any failed builds.*
>
> *At 9 AM, we have our daily Scrum standup where I coordinate with developers and QA. As the tech lead, I also do task assignment and sprint coordination for the DevOps team.*
>
> *The core of my day involves infrastructure work — writing Terraform code for infrastructure changes, improving CI/CD pipelines, handling deployment support, and doing code reviews for infrastructure changes. I also mentor 2-3 interns and provide technical guidance to the broader team.*

> *When incidents occur, I'm the primary coordinator — I handle triage, coordinate response, perform root cause analysis, and write post-incident reports. We follow ITIL-based incident management through Freshservice.*
>
> *I also dedicate time to infrastructure standards — defining deployment patterns, writing operational runbooks, and ensuring our practices are consistent across all application teams."*

Follow-up Questions

- **"How do you prioritize when you have a production incident AND a sprint deadline?"** → "Production always takes priority. I communicate the delay to stakeholders immediately, delegate sprint tasks where possible, resolve the incident, complete the RCA, and then re-plan the sprint work."
- **"How many deployments do you handle per day?"** → "We moved from bi-weekly releases to multiple daily deployments. On average, 3-5 deployments per day across all applications."

Chapter 4: Production Support

Probability: 🟠 80% — Frequently Asked

What Infosys Wants to Hear

Infosys is a consulting company. Their clients expect **24/7 production stability**. They need people who: - Can handle production on-call - Know how to triage and escalate - Understand SLAs and SLOs - Can write RCAs

Production Support Framework

```
INCIDENT SEVERITY MATRIX
┌──────────┬──────────────────────┬──────────┬──────────┐
│ Severity │ Definition           │ Response │ Resolve  │
├──────────┼──────────────────────┼──────────┼──────────┤
│ SEV-1    │ Complete outage      │ 15 min   │ 1 hour   │
│ SEV-2    │ Major degradation    │ 30 min   │ 4 hours  │
│ SEV-3    │ Minor issue          │ 2 hours  │ 24 hours │
│ SEV-4    │ Low impact           │ Next day │ 72 hours │
└──────────┴──────────────────────┴──────────┴──────────┘
```

Interview Questions

Q1: "How do you handle an on-call situation at 2 AM?"

> ***Answer:*** *"First, I check the alert details — severity, affected services, and impact. For a SEV-1, I immediately join the incident bridge, assess the blast radius, and start triage. My first step is always to check monitoring dashboards — Grafana for metrics, Loki for logs — to understand what changed. I follow the runbook if one exists. If the issue requires application-level fixes, I loop in the on-call*

> *developer. Throughout, I update the incident channel with status. After resolution, I document the timeline and write a blameless RCA within 24 hours."*

Q2: "Describe your SLA management approach"

> ***Answer:*** *"We define SLOs for each service — availability (99.9%), latency (p99 < 500ms), error rate (< 0.1%). These are monitored through Prometheus with Alertmanager rules. When we approach SLO burn rate thresholds, we get early warnings. This proactive approach helps us maintain our 99.9% SLA across 15+ applications. Monthly, we review SLO compliance reports in Grafana dashboards and adjust alert thresholds based on trends."*

Q3: "What's the difference between SLA, SLO, and SLI?"

Term	Definition	Your Example
SLI (Service Level Indicator)	The actual measured metric	Request latency p99 = 320ms
SLO (Service Level Objective)	The target you set internally	p99 latency should be < 500ms
SLA (Service Level Agreement)	The contractual promise to clients	99.9% uptime guaranteed

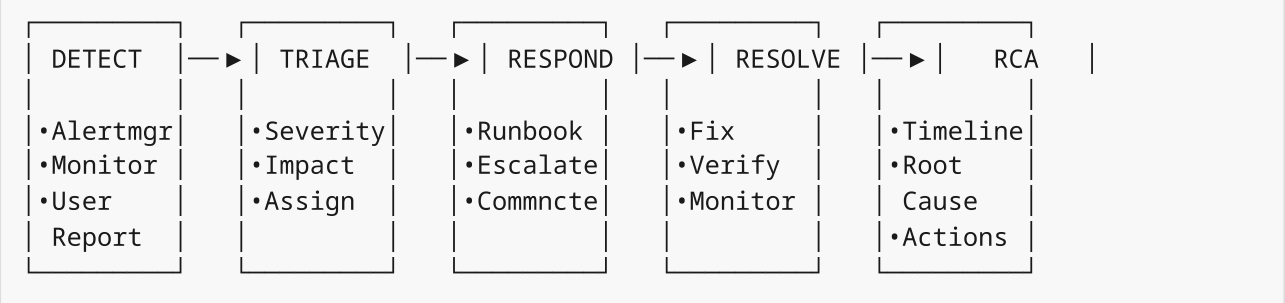
Production Tips from Real Experience

1. **Always have runbooks** — When you're debugging at 2 AM, you don't want to think. You want to follow steps.
2. **Pre-write incident response templates** — Slack/Teams message templates save 5 minutes of typing during crises
3. **Test your monitoring** — If Prometheus goes down, who monitors the monitor? Always have meta-monitoring.
4. **Backup your backups** — Cross-region replication. Test restores quarterly.

Chapter 5: Incident Handling

Probability: 🟠 80% — Frequently Asked (especially at consulting companies)

The Incident Lifecycle



STAR Format Example — Production Incident

Q: "Tell me about a critical production incident you handled."

> ***Situation:*** *"We had a PostgreSQL primary node failure during peak hours that affected 5 application services and 2,000+ active users."*
>
> ***Task:*** *"As the incident coordinator, I needed to restore database service, minimize downtime, and ensure data integrity."*
>
> ***Action:*** *"I immediately checked our Patroni cluster status and found the primary had OOM-killed due to a runaway query. Patroni had NOT automatically promoted the replica because the DCS (etcd) had a stale leader key. I manually triggered a controlled failover using `patronictl switchover`, verified data consistency by checking WAL replay status on the new primary, then identified and killed the runaway query. I updated the connection strings, drained the old primary, and restarted it as a replica."*
>
> ***Result:*** *"Total downtime was 8 minutes. We prevented data loss due to synchronous replication being enabled. I wrote an RCA that led to three improvements: 1) Added query timeout limits, 2) Configured OOM score adjustments for Patroni processes, 3) Updated our runbook with this specific failover scenario."*

RCA Template

```
# Root Cause Analysis — [Incident Title]
**Date:** YYYY-MM-DD
**Severity:** SEV-X
**Duration:** XX minutes
**Impact:** [Users/services affected]

## Timeline
| Time  | Event                              |
|-------|------------------------------------|
| 14:30 | Alert fired: PostgreSQL primary down |
| 14:32 | On-call acknowledged                |
| 14:35 | Triage: Patroni failover stuck      |
| 14:38 | Manual switchover initiated         |
| 14:40 | New primary accepting connections   |
| 14:42 | Application services recovered      |

## Root Cause
PostgreSQL OOM-killed due to unbounded analytical query.
Patroni auto-failover blocked by stale DCS leader key.

## Contributing Factors
1. No query timeout configured
2. No OOM score adjustment for critical processes
3. Runbook did not cover manual Patroni failover

## Action Items
| Action                   | Owner  | Deadline   | Status |
```

```
|--------------------------|--------|------------|--------|
| Configure statement_timeout| Aniket | 2025-04-01 | Done   |
| Set OOM score adj         | Aniket | 2025-04-01 | Done   |
| Update runbook            | Aniket | 2025-04-03 | Done   |

## Lessons Learned
- Synchronous replication prevented data loss
- Automated failover must be tested monthly
```

Common Interview Questions on Incident Handling

Question	Key Points to Cover
"How do you ensure incidents don't repeat?"	RCA → Action items → Runbook updates → Automated monitoring → Quarterly reviews
"What's your escalation process?"	Severity-based → On-call → Team lead → Manager → VP (if SEV-1 > 1 hour)
"How do you communicate during incidents?"	Dedicated channel, regular status updates (every 15 min for SEV-1), stakeholder notifications
"What tools do you use for incident management?"	Freshservice for ITSM, Slack/Teams for communication, Grafana/Prometheus for monitoring, Confluence/Wiki for RCA documentation
"What's a blameless post-mortem?"	Focus on SYSTEMS, not PEOPLE. Ask "what failed" not "who failed". Goal is learning and improvement, not blame.

Best Practices

1. **Never deploy on Fridays** — Unless you have weekend on-call coverage
2. **Always have a rollback plan** — Before every deployment, know how to undo it
3. **Test your DR** — A DR plan that's never been tested is just a wish
4. **Automate toil** — If you're doing it manually more than twice, automate it
5. **Document everything** — Your future 2 AM self will thank you